Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n)\, K(m,n)$$

where X: Input Image, K: Kernel (Filter), Y: Feature Map.

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where N = Input Size, F = Filter Size, P = Padding, S = Stride.

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map.

1. Identity Kernel Preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose: Preserves the original image; used for understanding convolution.

2. Sobel-X Kernel (Vertical Edge Detection)

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications: Object detection, lane detection, face recognition.

3. Sobel-Y Kernel (Horizontal Edge Detection)

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications: Text detection, boundary extraction, medical image analysis.

4. Laplacian Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications: Edge enhancement, satellite image analysis, medical imaging.

5. Sharpening Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications: Image enhancement, OCR, face recognition.

6. Box Blur Kernel

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications: Noise removal, image smoothing, preprocessing.

7. Gaussian Blur Kernel

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$$

Applications: Noise reduction, computer vision preprocessing, feature extraction.

8. Emboss Kernel

$$\begin{bmatrix}-2 & -1 & 0\\ -1 & 1 & 1\\ 0 & 1 & 2\end{bmatrix}$$

Applications: Texture analysis, artistic image effects.

9. Outline Detection Kernel

$$\begin{bmatrix}-1 & -1 & -1\\ -1 & 8 & -1\\ -1 & -1 & -1\end{bmatrix}$$

Applications: Image segmentation, boundary detection, shape extraction.

10. Motion Blur Kernel Simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix}1 & 0 & 0 & 0 & 0\\ 0 & 1 & 0 & 0 & 0\\ 0 & 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 0 & 1\end{bmatrix}$$

Applications: Motion simulation, image restoration, video processing.

3.2 Summary of Common Kernels

Table 1: Summary of common convolution kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges (all directions)	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4 Numerical Example 1: Convolution

Input image:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

First position: $1(1) + 2(0) + 4(0) + 5(1) = 6$
Second position: $2(1) + 3(0) + 5(0) + 6(1) = 8$
Third position: $4(1) + 5(0) + 7(0) + 8(1) = 12$
Fourth position: $5(1) + 6(0) + 8(0) + 9(1) = 14$

Feature Map:

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5 Numerical Example 2: Max Pooling

Feature map:

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter:

Window 1: $\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$ **Window 2:** $\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$

Window 3: $\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$ **Window 4:** $\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$

Output:

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6 Numerical Example 3: Parameter Calculation

Input Image: $32 \times 32 \times 3$

Convolution Layer: Filters = 16, Kernel = 3×3

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$$

Therefore, total trainable parameters = 448.

7 Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size: $32 \times 32 \times 3$

8 Experimental Procedure

Task 1 Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2 Implement a convolution layer. Compare Kernel = 3×3, 5×5, 7×7. Record feature map sizes.

Task 3 Study hyperparameters. Compare Stride = 1, Stride = 2, Padding = Same, Padding = Valid. Compute output dimensions.

Task 4 Visualize feature maps after the first convolution layer. Display at least eight feature maps.

Task 5 Compare Max Pooling and Average Pooling. Compare output size and accuracy.

Task 6 Construct the following CNN:

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

Train for: Optimizer – Adam, Epochs – 20, Batch Size – 32.

Task 7 Evaluate Accuracy, Precision, Recall, F1-score, Confusion Matrix, Classification Report.

9 Source Code

The following the code implementation of this experiment:
`https://github.com/D-Rockie/Deep-Learning-Lab-2026/blob/main/DLexp3.ipynb`

10 Mandatory Plots

Generate the following plots and write a 2–3 line inference for each.

1. Sample Images
2. Class Distribution
3. Training Accuracy
4. Validation Accuracy
5. Training Loss
6. Validation Loss
7. Feature Maps
8. Confusion Matrix

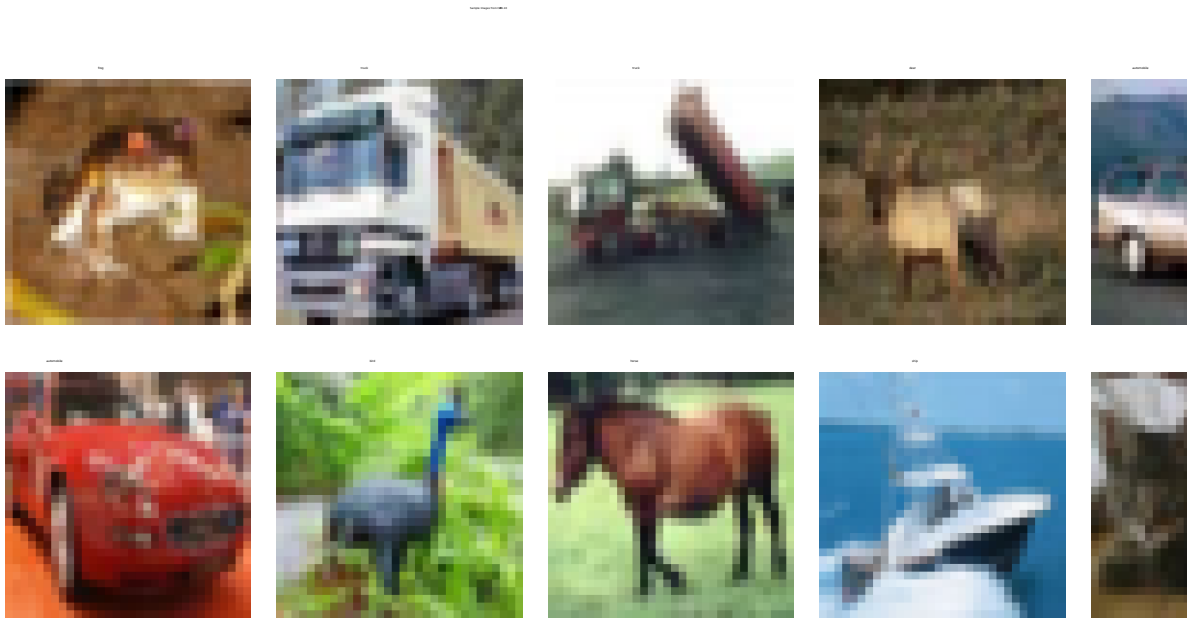

Figure 1: Sample images from the CIFAR-10 dataset.

Inference: The sample images demonstrate the visual diversity present within the CIFAR-10 dataset. Images belonging to the same class can have different colors, orientations, backgrounds, and object appearances, making automated classification challenging.

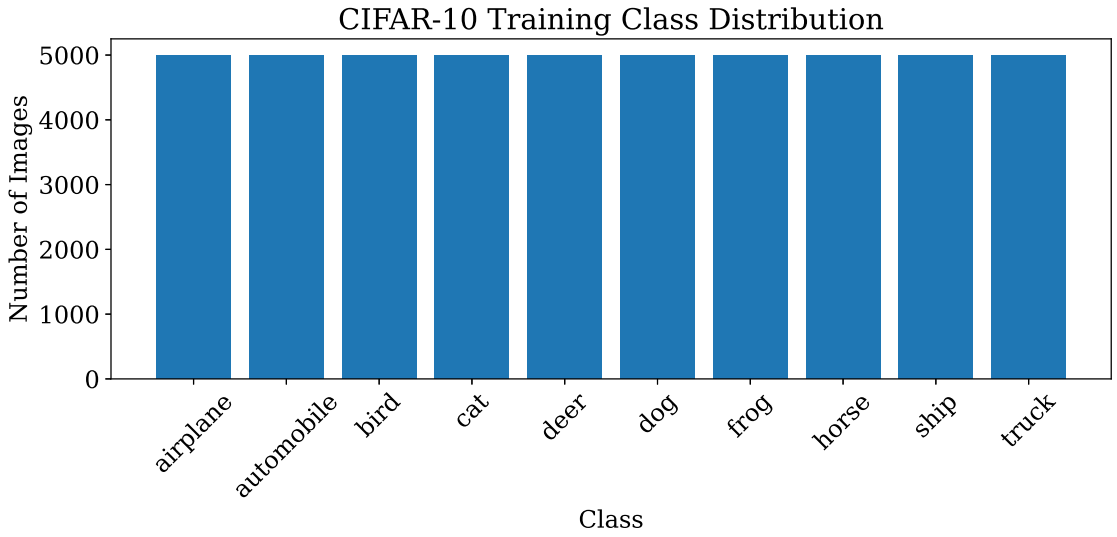

Figure 2: Class distribution of the CIFAR-10 training dataset.

Inference: The CIFAR-10 training dataset is approximately balanced, with each class containing nearly the same number of images. Therefore, substantial class imbalance is not expected to influence the classification performance.

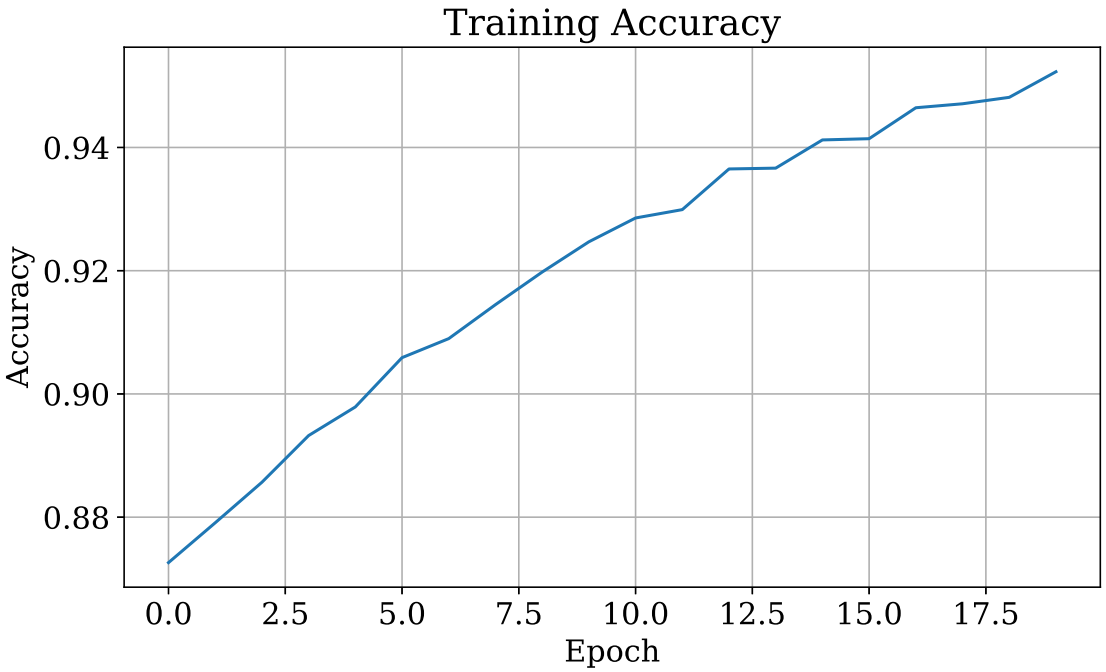

Figure 3: Training accuracy over 20 epochs.

Inference: The training accuracy generally increases as the number of epochs increases, indicating that the CNN progressively learns useful representations from the training data. The rate of improvement decreases as the model approaches convergence.

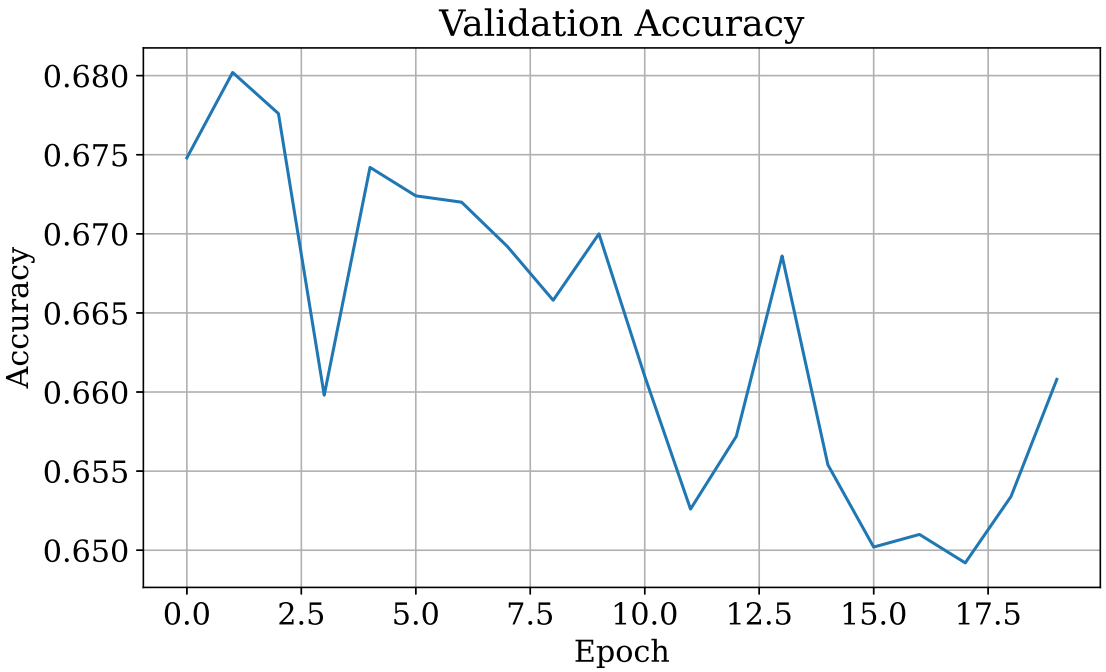

Figure 4: Validation accuracy over 20 epochs.

Inference: Validation accuracy indicates how effectively the trained CNN generalizes to unseen images. A stable increase suggests improved generalization, while a gap between training and validation accuracy may indicate overfitting.

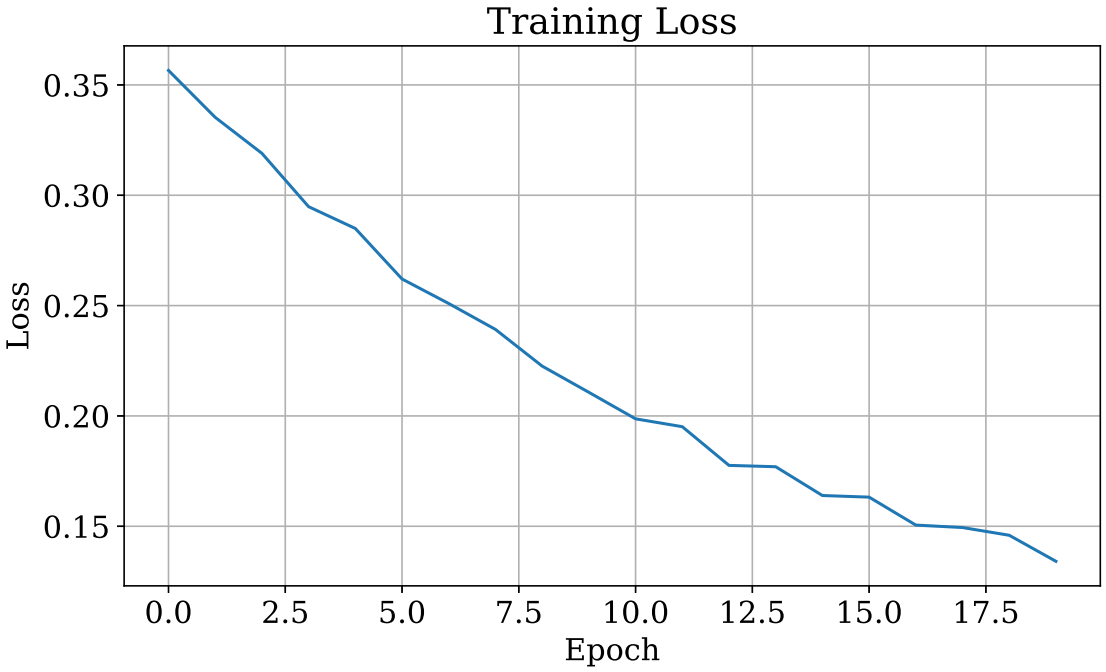

Figure 5: Training loss over 20 epochs.

Inference: The training loss is expected to decrease as the CNN learns to minimize classification errors on the training data. A decreasing loss indicates that the optimization process is successfully

updating the model parameters.

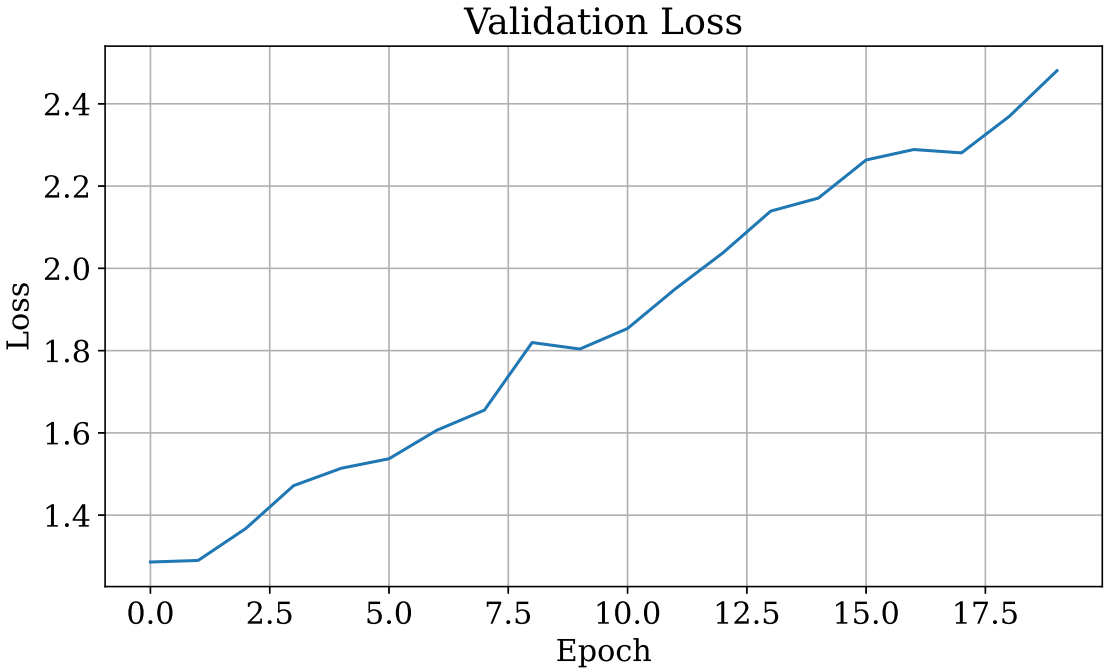

Figure 6: Validation loss over 20 epochs.

Inference: Validation loss reflects the model's performance on previously unseen data. If the validation loss begins increasing while training loss continues decreasing, it may indicate overfitting.

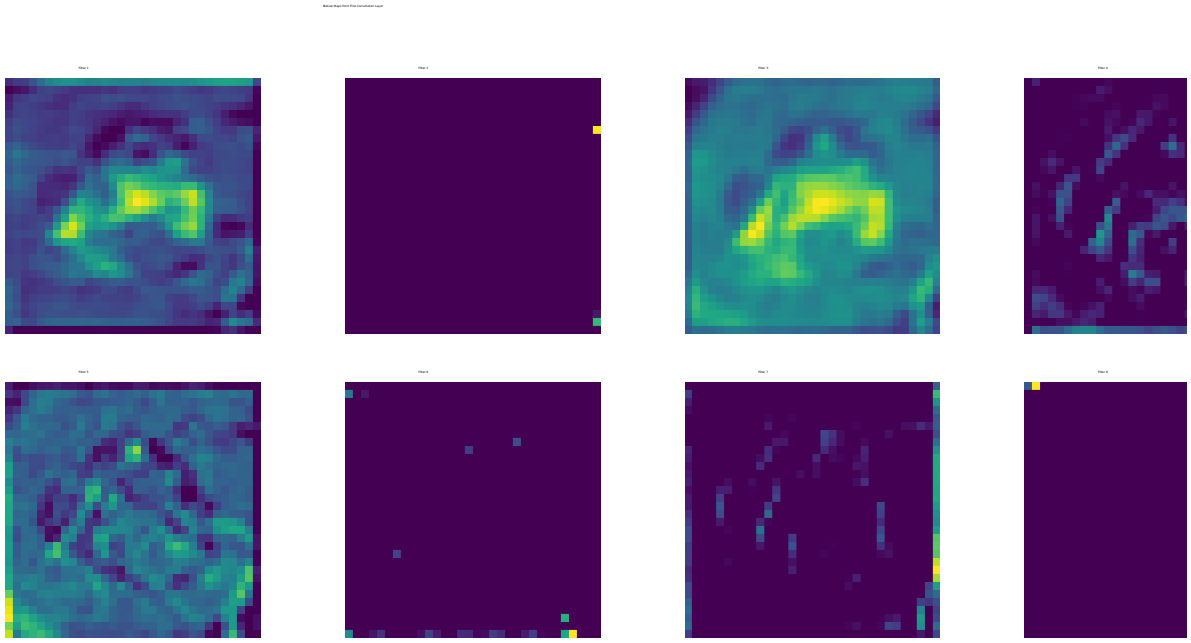

Figure 7: Feature maps generated by the first convolutional layer.

Inference: The eight feature maps show that different convolutional filters respond to different visual patterns in the input image. Since this is an early convolutional layer, the extracted representations

primarily correspond to low-level features such as edges, textures, and local patterns.

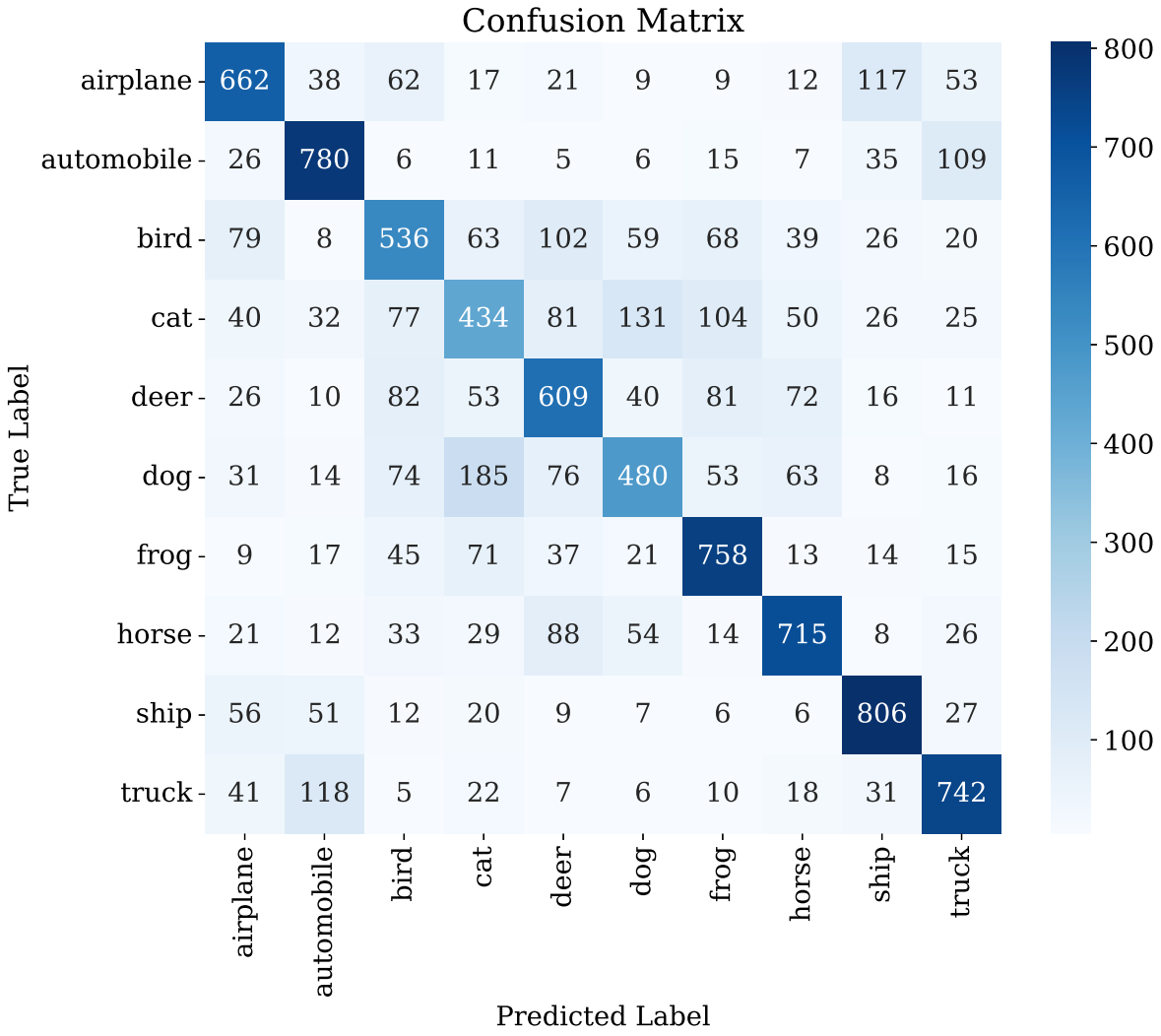

Figure 8: Confusion matrix of the CNN on the CIFAR-10 test dataset.

Inference: The diagonal entries represent correctly classified test images, while the off-diagonal entries represent misclassified samples. The confusion matrix helps identify pairs of classes that are difficult for the CNN to distinguish.

11 Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.
   Answer: Using $\frac{N - F + 2P}{S} + 1 = \frac{64 - 5 + 2(2)}{2} + 1 = \frac{63}{2} + 1 = 31.5 + 1$. Since the output dimension must be an integer, this is floored to give an output size of 32×32.
2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.
   Answer: Parameters $= (3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1{,}792$.
3. Compare ReLU and Sigmoid activation functions.
   Answer: ReLU, $f(x) = \max(0, x)$, outputs $[0, \infty)$, is cheap to compute, and avoids the vanishing-gradient problem since its gradient is constant (1) for positive inputs; its main drawback is the

"dying ReLU" problem, where neurons can become permanently inactive. Sigmoid, $f(x) = \frac{1}{1+e^{-x}}$, squashes outputs to $(0, 1)$ but saturates for large $|x|$, causing vanishing gradients in deep networks, and is computationally more expensive due to the exponential term. ReLU is generally preferred in hidden layers of deep CNNs, while sigmoid is typically reserved for output layers of binary classifiers.

4. Replace Max Pooling with Average Pooling and compare the results.
   Answer: This was implemented in the notebook. Both pooling variants were trained under identical conditions (Adam optimizer, 20 epochs, batch size 32) with the same parameter count (136,874), since pooling layers have no trainable parameters. The average-pooling model achieved a higher test accuracy (0.6810) than the max-pooling model (0.6522). Max pooling reached a much higher training accuracy (0.9523) but showed a larger training–validation gap, indicating heavier overfitting, whereas average pooling generalized slightly better on this dataset.
5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.
   Answer: This was implemented in the notebook. Increasing the filters from 16 to 64 increased the trainable parameters from 136,874 to 301,578 and improved the test accuracy from 0.6522 to 0.6895. This came at a significant computational cost: training time per epoch increased roughly threefold (from about 45–47 seconds to 130–140 seconds), since more filters require more convolution operations per layer. This illustrates the typical accuracy–computation trade-off in CNN design.

12 Results

Record

- Training Accuracy: 0.9523
- Testing Accuracy: 0.7003
- Precision: 0.6987
- Recall: 0.7003
- F1-score: 0.6980
- Number of Parameters: 136,874

13 Discussion

1. Why is convolution preferred over fully connected layers for images?
2. How does stride affect the feature map size?
3. What is the role of padding?
4. Why is pooling used?
5. How do feature maps represent image characteristics?
6. Why do CNNs require fewer parameters than MLPs?

14 References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.